# Memory Management and Garbage Collection Strategies in High-Level Quantum Programming Languages

Rajveer Chopra
Reg. No.: 24BCE0122
*School of Computer Science and Engineering (SCOPE)*
*Vellore Institute of Technology*
Vellore, Tamil Nadu, India
rajveer.singhchopra2024@vitstudent.ac.in

***Abstract*—High-level quantum languages let programmers allocate temporary qubits like local variables, but a live qubit cannot be copied, inspected or silently dropped, so classical garbage collection does not apply. This paper presents QRECLAIM, a compile-time garbage collector for qubits built on a linear intermediate representation. A forward basis-state abstract interpretation computes each ancilla's reversible trace, dependencies and uncomputation deadline. An uncomputation-aware extension of live-variable analysis then places each uncomputation at the earliest provably safe point. The same pass reclaims measured qubits by early measurement and reset and deletes ancillas that are never read, and a validator proves statically that every freed qubit is $|0\rangle$. An earliest-ready-first linear-scan allocator exposes the width–depth trade-off as one qubit budget. On six benchmark families (29 instances) QRECLAIM uses 53.8% fewer qubits than compilation without reuse, and its depth-neutral mode saves 37.7% with no increase in depth. In a bug-injection study the static checks reject all 175 faulty programs at 94.6% precision.**

***Index Terms*—quantum compilers, uncomputation, qubit reuse, garbage collection, data-flow analysis, register allocation**

## I. INTRODUCTION

Quantum processors offer tens to a few hundred qubits, so a program's *width* (the number of qubits it needs) decides whether it runs at all. Programs are increasingly written in high-level languages such as Silq [1], Qrisp [2] and Qurts [3], where temporary qubits (*ancillas*) are declared like local variables and returned automatically. Returning them safely and early is the quantum analogue of memory management.

Classical garbage collection does not carry over. A copying collector moves objects, but a quantum state cannot be copied [4]; a tracing collector inspects the heap, but measurement disturbs a qubit. Worse, a dead temporary is usually entangled with live data, and discarding it destroys interference. It can only be freed by *uncomputing* it, i.e. running the gates that produced it in reverse [5], and only while its inputs are unchanged. Mid-circuit measurement and reset offer a second route [6], [7]. A compiler therefore faces four design challenges: *safety* (never alias two states onto one qubit or leave entangled garbage), *placement* (where, between the last use and the next write to an input, to uncompute, given that ancillas computed from ancillas extend each other's lifetimes), *unification* (deciding uncomputation-based and measurement-based reclamation in one analysis), and *allocation* (reuse serialises parallel work, so fewer qubits can mean deeper circuits).

QRECLAIM answers these with compiler machinery adapted to quantum constraints. Its contributions are:

- a linear IR whose typing rules forbid aliasing and whose T-FREE rule is discharged by a static validator (§III-A);
- a basis-state abstract interpretation and an uncomputation-aware liveness analysis, written as data-flow equations with a legality condition and a soundness theorem (§III-B);
- a unified reclamation pass for uncomputation, early measurement and dead-garbage elimination, applied before and after gate lowering, plus an earliest-ready-first (ERF) allocator with a qubit budget (§III-C–III-D);
- an evaluation against no reuse, lexical reclamation, a strong baseline combining prior techniques and the Qrisp framework, plus a bug-injection study (§V).

## II. LITERATURE REVIEW

*High-level languages and uncomputation.* Silq [1] introduced safe automatic uncomputation: `qfree` and `const` annotations identify reversible computations and unchanged inputs, and the type system rejects unphysical programs. Qurts [3] carries affine types with lifetimes over to qubits, and a value dropped within its lifetime is uncomputed automatically. Qrisp [2] follows the Unqomp approach [8]: its compiler maps qubits of deleted variables onto free physical qubits, and a `workspace` option trades extra qubits for depth. Unqomp synthesises uncomputation on a circuit graph, Reqomp [9] trades recomputation for ancillas under a space budget, and SQUARE [10] decides at each call site whether to uncompute or keep garbage, using a cost model. All build on Bennett's compute–copy–uncompute construction [5].

*Reuse by measurement and reordering.* CaQR [6] reuses a qubit after its final measurement by inserting a reset, reporting resource reductions of up to 80%. DeCross *et al.* [7] solve measurement-based reuse exactly and greedily. Jiang [11]

recasts circuit-level recycling as matrix triangularisation and proves it NP-hard, which justifies heuristic reordering. Reichental *et al.* [12], closest to our scheduler, search for a topological order of a high-level program that maximises the reuse of uncomputed qubits.

*Compiler frameworks and static analysis.* Industrial frameworks such as Qiskit [13] compile through a pass manager over a DAG IR but treat qubits as fixed registers and never reclaim them. Quantum static analysis is mostly semantic: Yu and Palsberg [14] abstract states by projections on small qubit subsets to verify assertions in polynomial time. Our analysis is structural instead: it tracks reversible traces, dependencies and lifetimes, and it transfers live-variable analysis and linear-scan register allocation [15] to qubits. Two classical notions do not carry over: spilling is impossible (no copying), and freeing requires uncomputation or measurement.

*Research gaps.* **G1**: uncomputation-based reuse [2], [8]–[10] and measurement-based reuse [6], [7], [11] are separate passes at different abstraction levels. **G2**: reclamation legality is stated as type rules [1], [3] or circuit-graph conditions [8], not as data-flow facts that compose with liveness, scheduling and allocation. **G3**: static guarantees that freed qubits are clean, and diagnostics when no safe reclamation exists, are language-specific, and their precision is rarely measured.

## III. Proposed Model / Architecture

*System overview.* Fig. 1 shows the pipeline. Programs in an embedded front end become QR-IR, a straight-line list of instructions over logical qubits. A linear type check rejects aliasing. A forward basis-state analysis and a backward liveness analysis drive the level-1 reclamation pass, and a validator proves each `free` it inserts safe. Multi-controlled gates are then lowered to Toffoli chains, and the same pass and validator reclaim the helper qubits this creates (level 2). A pressure-aware scheduler reorders commuting instructions, an ERF linear-scan allocator maps logical to physical qubits, and code is emitted for Qiskit or as OpenQASM 2.

### A. QR-IR and static semantics

A program is a sequence $P = \iota_0 \dots \iota_{N-1}$ of

$$\iota ::= \texttt{alloc}\ q \mid \texttt{free}\ q \mid g^{\theta}(\bar{c}^{\bar{\pi}}; \bar{t}) \mid \texttt{measure}\ q \to c \mid \texttt{reset}\ q$$

where $\bar{c}$ are controls with polarities $\pi \in \{+,-\}$, $\bar{t}$ are targets and $\theta$ are angles. CX, Toffoli and $k$-controlled X are the same operation `x` with $|\bar{c}| = 1, 2, k$. Qubits are *data*, *ancillas* (reclaimed by the compiler) or *helpers* (made by lowering). Gates are classed by their action on basis states: $G_{\mathrm{qf}} = \{\texttt{x}, \texttt{swap}\}$ permute it, $G_{\mathrm{dg}} = \{\texttt{z}, \texttt{s}, \texttt{t}, \texttt{rz}, \texttt{p}\}$ (and inverses) are diagonal, and $G_{\mathrm{gen}} = \{\texttt{h}, \texttt{rx}, \texttt{ry}\}$ may create superposition. The write set $W(\iota)$ holds the targets of $G_{\mathrm{qf}} \cup G_{\mathrm{gen}}$ gates and of `reset`; controls and diagonal gates never write. QR-IR has no classically controlled operations, so after inlining and unrolling a program is one basic block. Programmers write only the forward computation and never uncompute or free.

TABLE I
Transfer functions $f_\iota$ ($\iota$ at position $i$)

| Instruction $\iota$ | Effect |
|---|---|
| `alloc` $q$; `reset` $q$ | $\sigma(q)\leftarrow\mathbf{0}$; $Tr(q)\leftarrow\epsilon$; $Dep(q)\leftarrow\emptyset$; $D(q)\leftarrow\infty$ |
| $g(\bar{c};t)$, $g \in G_{\mathrm{qf}}$, $\sigma(t) \in \{\mathbf{0}, \mathbf{B}\}$, $D(t) = \infty$ | $\sigma(t)\leftarrow\mathbf{B}$; $Tr(t)\leftarrow Tr(t)\cdot\iota$; $Dep(t)\leftarrow Dep(t)\cup\bar{c}$ |
| other qfree gate; $g \in G_{\mathrm{gen}}$ | $\sigma(t)\leftarrow\mathbf{T}$ for every target $t$ |
| $g \in G_{\mathrm{dg}}$; `measure` $q$ | no change; $\sigma(q)\leftarrow\mathbf{M}$ |
| any $\iota$, each $w \in W(\iota)$ | for all $a$ with $w \in Dep(a)$, $D(a) = \infty$: $D(a)\leftarrow i$ |

Judgements $\Delta \vdash \iota \dashv \Delta'$ thread a context of live qubits and their abstract states:

$$\frac{q \notin \mathrm{dom}(\Delta)}{\Delta \vdash \texttt{alloc}\ q \dashv \Delta, q{:}\mathbf{0}}\ \text{(T-Alloc)}$$

$$\frac{\vdash_{XOR} q \text{ clean}}{\Delta \vdash \texttt{free}\ q \dashv \Delta \setminus q}\ \text{(T-Free)}$$

$$\frac{q \in \mathrm{dom}(\Delta)}{\Delta \vdash \texttt{measure}\ q \dashv \Delta[q \mapsto \mathbf{M}]}\ \text{(T-Use)}$$

$$\frac{\bar{c}\bar{t} \text{ pairwise distinct} \quad \bar{c}\bar{t} \subseteq \mathrm{dom}(\Delta)}{\Delta \vdash g(\bar{c};\bar{t}) \dashv \Delta[\bar{t} \mapsto \tau_g]}\ \text{(T-Gate)}$$

T-Use also covers `reset`; $\tau_g$ is given by Table I. QR-IR has no copy instruction, so no program can clone a state [4]. The rules guarantee what a qubit manager needs instead: no operation uses one qubit in two roles, and a qubit is freed, and later handed out again, only when it is provably $|0\rangle$ (§III-C). After allocation a physical check confirms that no physical qubit hosts two live logical qubits.

### B. Data-flow analyses

*Basis-state abstract interpretation (forward).* Each qubit carries a state in the lattice $\bot \sqsubseteq \mathbf{0} \sqsubseteq \mathbf{B} \sqsubseteq \mathbf{T}$, $\bot \sqsubseteq \mathbf{M} \sqsubseteq \mathbf{T}$. The states mean unallocated, exactly $|0\rangle$, a basis value that is a reversible function of other qubits, measured, and possibly superposed. Each qubit also carries a *trace* $Tr(q)$ (the qfree gates that computed it), a *dependency set* $Dep(q)$ (their controls) and a *deadline* $D(q)$, the first later position that writes some $d \in Dep(q)$. With $\mathrm{IN}[0] = \bot^{Q}$, $\mathrm{OUT}[i] = f_{\iota_i}(\mathrm{IN}[i])$ and $\mathrm{IN}[i{+}1] = \mathrm{OUT}[i]$, one forward sweep computes all facts (Table I).

*Liveness (backward).* With $DEF(\iota) = \{q\}$ for `alloc` $q$ and $USE(\iota)$ the operands of every other instruction,

$$LIVE_{in}(\iota_i) = USE(\iota_i) \cup \big(LIVE_{in}(\iota_{i+1}) \setminus DEF(\iota_i)\big), \quad (1)$$

$$L(q) = \max\{\, i : q \in USE(\iota_i) \,\}. \quad (2)$$

*Uncomputation-aware liveness.* Inserting the uncomputation $U(a)$ of ancilla $a$ at $P(a)$ reads every $d \in Dep(a)$ again, so the last use of a dependency can move later. For the set $R$

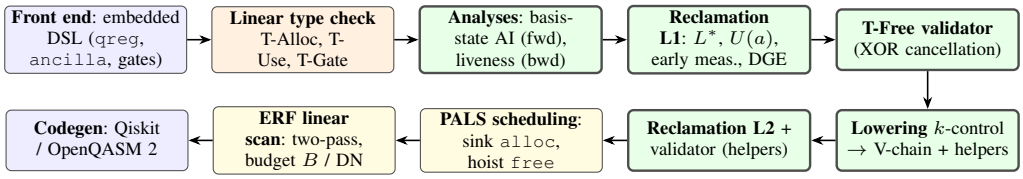



Fig. 1. The QRECLAIM pipeline. Bold-bordered stages form the compile-time garbage collector; the same reclamation pass and validator run at two IR levels.

of ancillas to uncompute, the effective last use is the least fixpoint of

$$L^*(q) = \max\big(L(q),\ \max\{P(a) : a \in R,\ q \in Dep(a)\}\big), \tag{3}$$

$$P_{\text{eager}}(a) = L^*(a), \tag{4}$$

$$P_{\text{lazy}}(a) = \min\big(end(scope(a)),\ D(a){-}1\big),$$

where the lazy (lexical) placement is additionally capped by $P_{\text{lazy}}(b)$ for $b \in Dep(a) \cap R$. A placement is *legal* iff

$$L^*(a) \ \le\ P(a) \ <\ D(a) \qquad \text{for every } a \in R. \tag{5}$$

QRECLAIM places eagerly. If (5) fails, the program is rejected with a diagnostic naming the instruction that overwrote the input. The fixpoint is solved round-robin; $P$ only grows and is bounded by $N$, so it terminates within $h{+}1$ sweeps for a dependency DAG of height $h$ (two sweeps on every benchmark).

*Theorem 1 (Sound uncomputation):* Suppose every $a \in R$ satisfies (5) and $\sigma(a) = \mathbf{B}$, let $U(a) = Tr(a)_k^{-1} \cdots Tr(a)_1^{-1}$, and order blocks at a shared point so that readers of $q$ precede the block that frees $q$. Then every freed ancilla is $|0\rangle$ and unentangled, and the final state of the other qubits does not depend on which legal $P(a)$ is chosen.

*Proof sketch:* Since $P(b) \le L^*(a) \le P(a)$ whenever $a \in Dep(b)$, and resets of $d \in Dep(a)$ are placed after $P(a)$, nothing writes $Dep(a)$ between the trace and $U(a)$. In each computational-basis branch $|d\rangle$ of $Dep(a)$, every trace gate therefore sees the same controls when applied and when reversed. All other operations only read $a$ (a write would make $\sigma(a) = \mathbf{T}$), so $U(a)$ maps $|d\rangle|f(d)\rangle \mapsto |d\rangle|0\rangle$, and by linearity $a$ factors out as $|0\rangle$. Within the window, the other instructions and $U(a)$ are block-diagonal in that basis and act on disjoint qubits, so they commute. ∎

Similarly, a qubit whose final operation is a measurement may be measured right after its previous use, then reset and freed. This preserves the outcome distribution, because measurement commutes with operations on other qubits and with operations that use it only as a control.

### C. Reclamation and validation

Algorithm 1 summarises the pass. *Dead-garbage elimination* (DGE) deletes ancillas whose only uses are their own trace gates, together with their computation. A gate with $k \ge 3$ controls is lowered to a V-chain of $k{-}2$ helpers plus one Toffoli; lowering emits only the compute half, and rerunning the pass on helpers produces the uncompute half. Level 1 must run *before* lowering: there an ancilla computed by a 3-controlled gate depends only on its inputs, whereas after lowering it would depend on a helper and keep it alive.

**Algorithm 1** RECLAIM$(P, K)$, eager mode

1: $(\sigma, Tr, Dep, D) \leftarrow$ BASISAI$(P)$; $L \leftarrow$ LIVENESS$(P)$
2: delete ancillas $a \in K$ with $uses(a) = Tr(a)$ and their traces ▷ DGE
3: **if** $\exists a \in K : \sigma(a) = \mathbf{T}$ **then error** "unreclaimable ancilla"
4: $R \leftarrow \{a \in K : \sigma(a) = \mathbf{B}\}$
5: **repeat for** $a \in R$: $P(a) \leftarrow \max(L(a), \max\{P(b) : a \in Dep(b)\})$
6: **until** no $P(a)$ changed
7: **check** Eq. (5) for all $a \in R$, **else error** naming $D(a)$
8: **for all** $q$ whose final operation is `measure` **do**
9: move it after the previous use of $q$ and every $P(b)$ with $q \in Dep(b)$; append `reset` $q$; `free` $q$
10: insert $U(a)$; `free` $a$ after $\iota_{P(a)}$ for all $a \in R$, readers before writers

*Static validation of frees.* QRECLAIM validates its own output (translation validation). Consider the operations on $a$ since its last `alloc` or `reset`, up to a `free` $a$. They must be X-type writes $a \oplus\!= \bigwedge(\bar{c}^{\bar{\pi}})$, uses of $a$ as a control, or diagonal gates. Every control pattern must occur an even number of times, with no write to its controls between its first and last occurrence. Each term is then XORed in an even number of times with equal control values on every branch, so $a$ is $|0\rangle$ and unentangled, which discharges T-FREE. The validator runs after both reclamation levels in every compilation.

### D. Scheduling and allocation

Gates on disjoint qubits commute, so any topological order of the per-qubit precedence DAG is equivalent. Pressure-aware list scheduling (PALS) emits ready instructions in order of their effect on the number of live qubits: `free` ($-1$), then other operations (0), then `alloc` ($+1$). This sinks allocations and hoists reclamation, the program-level counterpart of reordering for reuse [6], [11], [12]. Each logical qubit then occupies one interval, so interference forms an interval graph.

*Proposition 1:* For a fixed order with at most $\omega$ qubits allocated at once, linear scan that reuses a free qubit whenever one exists uses exactly $\omega$ physical qubits, the minimum [15]. A second pass with budget $B = \omega$ that takes fresh qubits first never fails.

On reuse, the allocator takes the pooled qubit whose last operation finishes earliest in the ASAP schedule (earliest-ready-first, via a min-heap). This minimises the false dependency. QRECLAIM-min uses $B = \omega$. QRECLAIM-DN (depth-neutral) scans $B$ upwards for the smallest budget no deeper than the no-reuse circuit; a scan is needed because depth is not

TABLE II
BENCHMARK FAMILIES (29 INSTANCES)

| ID | Program | Sizes | Stresses |
|---|---|---|---|
| BV | Bernstein–Vazirani, $n$-bit secret | $n$=8..128 | measured data |
| HS | hidden shift, one temporary per product | $n$=4..64 | flat temporaries |
| RC | repetition code, $r$=$d$ rounds, fresh syndrome qubits | $d$=3..11 | mid-circuit meas. |
| ADD | ripple-carry adder, carries in ancillas | $n$=4..64 | chain, DGE |
| GRV | Grover on planted 3-SAT, $m$=$2n$ clause ancillas | $n$=4..10 | oracle, lowering |
| QAOA | MaxCut layer on an $n$-cycle via $t$=$x_i \wedge x_{i+1}$ | $n$=4..64 | shared inputs |

monotone in $B$. Analyses and checks are $O(N)$, scheduling and allocation $O(N \log N)$.

## IV. IMPLEMENTATION / EXPERIMENTAL SETUP

*Toolchain.* QRECLAIM is about 1,500 lines of Python 3.13. We use a custom pass pipeline instead of LLVM or GCC because reclamation needs linear qubit values, qubit kinds and uncomputation, none of which LLVM IR models. Code generation targets Qiskit 2.5.2 [13], which also lowers every output to $\{\mathrm{CX}, U\}$ (optimisation level 1) so that all configurations are measured alike. An exact NumPy simulator branches on mid-circuit measurements and asserts at every `alloc` and `free` that the qubit is $|0\rangle$, and Qiskit Aer 0.17.2 serves as a cross-check. One script regenerates every table, figure and number.

*Benchmarks and configurations.* Table II lists the six families. Temporaries are allocated freely and never uncomputed by hand; QAOA's consecutive temporaries share an input. We compare:

- **NR**: correct uncomputation, no reuse;
- **SCOPE**: lazy (lexical) placement with stack (LIFO) reuse;
- **SB**, a strong baseline: lazy placement with CaQR-style early measurement, PALS and ERF, which isolates what eager placement and DGE add;
- **QRECLAIM-min** and **QRECLAIM-DN**;
- the external framework **Qrisp** 0.9.9, run on the same programs.

Metrics are width (memory footprint), CX count (code size), depth after lowering, resets and compile time. We also report an ASAP duration under an *assumed* latency model (1q gate $= 1$, CX $= 2$, measure and reset $= 10$).

*Bug injection.* Seven mutation operators inject single faults into six small benchmarks (237 mutants). Ground truth is exact: an ancilla is reclaimable if some insertion point after its last use returns it to $|0\rangle$ when its trace is reversed, for *every* basis input. Inputs are purified with reference qubits, so one simulation covers all inputs. A mutant is faulty if any ancilla is not reclaimable.

## V. RESULTS & DISCUSSION

*Correctness.* We simulated exactly every output of width $\leq 20$ with at most 1,024 measurement branches: 59 of the 145 compiled outputs, plus 35 of 36 small verification programs. All match the analytically expected distribution, with total variation distance $\leq 1.3 \times 10^{-14}$ and every run-time $|0\rangle$ assertion holding; Aer agrees within sampling noise ($\leq 0.027$). Statically, the validator discharged T-FREE for all 1944 frees in the QRECLAIM-min outputs.

*Memory footprint, code size and depth.* Averaged over 29 instances, QRECLAIM-min uses 53.8% fewer qubits than NR (geometric-mean width ratio 0.323), 42.8% fewer than SCOPE and 8.3% fewer than SB (Fig. 2, Table III). For BV and RC, early measurement (plus PALS for BV) reduces width to 2 and $d$+1, whatever the number of rounds. On HS and GRV, SB matches QRECLAIM because PALS hoists lazily placed uncomputations. QAOA's temporaries share inputs, so a lazily placed uncomputation cannot be hoisted past the next computation; only eager placement reclaims them ($n$+1 qubits against $2n$ for NR, SCOPE and SB, at *identical* depth). On ADD the carries form a chain ($L^*(c_i) \geq P(c_{i+1})$), so placement cannot help, and DGE removes the unused carry-out (one qubit, 22 CX at $n$=64). Reclamation adds no gates, because every configuration must uncompute for correctness; the extra operations are resets, one per reuse of a measured qubit. Width is bought with depth: QRECLAIM-min is 2.55× deeper than NR (geometric mean).

*Width–depth trade-off.* As the budget grows, depth reaches the NR depth at 33% (BV), 51% (GRV), 59% (RC) and 75% (HS) of the NR width. QRECLAIM-DN finds this point automatically and saves 37.7% of the qubits with no depth increase on any instance (maximum ratio 1.0). Under the slow-reset latency model, QRECLAIM-DN takes 0.819–2.194 times the duration of NR. The upper end comes from BV and RC, whose remaining resets cost time that unit depth does not show, so such hardware should select the budget with the latency model.

*Leave-one-out ablation* (Table IV). Early measurement is essential for BV and RC (2→33 and 8→49 qubits without it), PALS for BV, eager placement for QAOA (17→32) and DGE for ADD (47→48). ERF and the two-pass allocation leave width unchanged but cut depth (GRV: 1094 with LIFO, 943 without the second pass, 825 in full). On BV, HS, RC and GRV, removing eager placement changes nothing because PALS hoists lazy uncomputations; eager placement matters exactly when temporaries share inputs.

*External baseline* (Table V; all outputs verified by simulation). Qrisp reaches the *same* width on HS and QAOA, which confirms that dependency-aware reclamation is what a mature framework also achieves. It is narrower on GRV and ADD and shallower everywhere. The difference lies in gate synthesis, not reclamation: Qrisp builds multi-controlled gates without helper qubits and uses cheaper phase-tolerant Toffoli pairs, whereas QRECLAIM lowers to Barenco V-chains. Qrisp recycles a measured variable when the programmer deletes it, whereas QRECLAIM infers measurement-based reuse automatically (BV, RC), checks it statically and controls it through one budget.

*Static checks* (Table VI). All 175 faulty mutants were rejected (recall 100.0%), and no accepted mutant failed a run-time assertion. Precision is 94.6%: the 10 false alarms write a

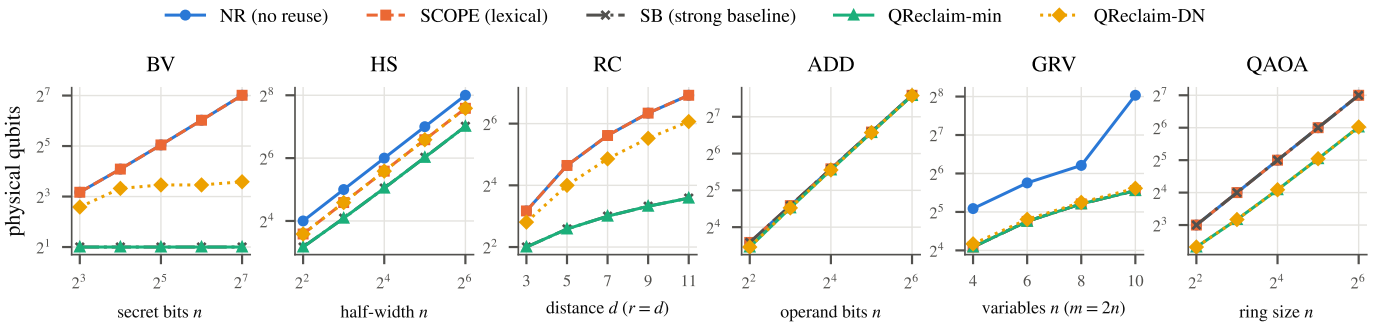



Fig. 2. Physical qubits (log scale) versus problem size. SB coincides with QRECLAIM-min except on ADD (DGE) and QAOA (eager placement); SCOPE coincides with NR on BV and RC.

TABLE III
RESULTS AT THE LARGEST SIZE OF EACH FAMILY: WIDTH, DEPTH, CX, RESETS (QRECLAIM-MIN/QRECLAIM-DN), QRECLAIM COMPILE TIME

| | Width (physical qubits) | | | | | Depth | | | | CX | | Resets | Compile |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Benchmark | NR | SCOPE | SB | QRECLAIM-min | QRECLAIM-DN | NR | SB | QRECLAIM-min | QRECLAIM-DN | NR | QRECLAIM-min | min/DN | ms |
| BV-128 | 129 | 129 | 2 | **2** | 12 | 67 | 447 | 447 | 67 | 64 | 64 | 127/117 | 6.6 |
| HS-64 | 256 | 192 | 129 | **129** | 192 | 44 | 1271 | 1271 | 44 | 1536 | 1536 | 31/0 | 15.3 |
| RC-11 | 121 | 121 | 12 | **12** | 67 | 42 | 405 | 405 | 42 | 220 | 220 | 109/54 | 8.3 |
| ADD-64 | 192 | 192 | 192 | **191** | 191 | 2962 | 2962 | 2935 | 2935 | 2267 | 2245 | 0/0 | 10.8 |
| GRV-10 | 262 | 47 | 47 | **47** | 49 | 2455 | 2557 | 2557 | 2437 | 3060 | 3060 | 0/0 | 13.5 |
| QAOA-64 | 128 | 128 | 128 | **65** | 65 | 1157 | 1157 | 1157 | 1157 | 768 | 768 | 31/31 | 6.0 |

TABLE IV
LEAVE-ONE-OUT ABLATION: WIDTH/DEPTH (BV-32, HS-16, RC-7, ADD-16, GRV-8, QAOA-16)

| Configuration | BV | HS | RC | ADD | GRV | QAOA |
|---|---|---|---|---|---|---|
| SCOPE | 33/21 | 48/44 | 49/26 | 48/706 | 37/1094 | 32/293 |
| SB | 2/117 | 33/371 | 8/149 | 48/706 | 37/825 | 32/293 |
| QRECLAIM (full) | 2/117 | 33/371 | 8/149 | 47/679 | 37/825 | 17/293 |
| −eager | 2/117 | 33/371 | 8/149 | 47/679 | 37/825 | 32/293 |
| −early-meas | 33/21 | 33/581 | 49/26 | 47/679 | 37/825 | 17/293 |
| −PALS | 33/21 | 33/581 | 8/158 | 47/679 | 37/825 | 17/293 |
| −DGE | 2/117 | 33/371 | 8/149 | 48/706 | 37/825 | 17/293 |
| −ERF (LIFO) | 2/117 | 33/671 | 8/167 | 47/679 | 37/1094 | 17/377 |
| −two-pass | 2/117 | 33/371 | 8/158 | 47/679 | 37/943 | 17/293 |

TABLE V
QRECLAIM-MIN VS. QRISP: WIDTH/DEPTH/CX AFTER THE SAME LOWERING

| Program | QRECLAIM-min | Qrisp | Qrisp (workspace 2) |
|---|---|---|---|
| HS-16 | 33/371/384 | 33/97/192 | 35/74/192 |
| QAOA-16 | 17/293/192 | 17/72/96 | 19/53/96 |
| GRV-6 | 27/663/588 | 18/377/258 | 20/315/250 |
| ADD-8 | 23/303/229 | 22/136/105 | 24/136/105 |

TABLE VI
BUG INJECTION: MUTANTS, FAULTY (GROUND TRUTH), TP, FN, FP, TN

| Operator | $n$ | faulty | TP | FN | FP | TN |
|---|---|---|---|---|---|---|
| alias an operand | 40 | 40 | 40 | 0 | 0 | 0 |
| overwrite control (early) | 40 | 31 | 31 | 0 | 3 | 6 |
| overwrite control (late) | 40 | 36 | 36 | 0 | 4 | 0 |
| flip and restore control | 20 | 17 | 17 | 0 | 3 | 0 |
| reset a control | 40 | 34 | 34 | 0 | 0 | 6 |
| H on ancilla | 17 | 17 | 17 | 0 | 0 | 0 |
| overwrite after last use | 40 | 0 | 0 | 0 | 0 | 40 |
| total | 237 | 175 | 175 | 0 | 10 | 52 |

dependency that later code restores, or that no remaining trace gate reads, which the first-write deadline $D(a)$ cannot see. Overwriting a control *after* the last use was always accepted, because eager placement uncomputes first. Diagnostics name the cause, e.g. "cannot uncompute ancilla t0: its control x0 is modified by `h x0` at [5] while t0 is still needed at [6]".

*Compilation time and threats to validity.* Every main instance compiles in at most 42.4 ms. On inputs scaled to 26,594 instructions, compile time grows with log–log slope 1.03 (median 10.3 $\mu$s per instruction), within 0.5–6.5 times Qiskit's own level-1 transpilation. Several threats remain. Depth assumes all-to-all connectivity. SCOPE and SB model prior strategies rather than re-implementing specific tools; Qrisp is the one external tool. The benchmarks are small to medium. Beyond about 20 qubits, guarantees rest on the static validator and checks instead of simulation.

## VI. CONCLUSION & FUTURE WORK

This case study treated qubit reclamation as compiler memory management. A linear IR with static semantics, a basis-state abstract interpretation and uncomputation-aware liveness decide where reclamation is legal. A unified pass inserts uncomputation, early measurement and resets, its output is statically validated, and an ERF linear scan exposes the width–depth trade-off through one budget. QRECLAIM reduced width by 53.8% on average and by 37.7% at no depth cost. It beat a strong baseline built from prior techniques where temporaries share inputs, matched Qrisp's width on two of four shared

families, and its static checks found every injected fault. Three extensions are technically meaningful:

1) **Recomputation-aware placement.** Adding optional recomputation edges to Eq. (3) would let chains such as ADD's carries be reclaimed early. Placement then becomes a min-cost reversible pebbling problem under budget $B$ [9]: ILP for small dependency DAGs, a fixpoint heuristic otherwise.
2) **Restoration-aware deadlines and synthesis.** Generalising $D(a)$ to windows that reopen when the XOR test of §III-C shows a dependency restored would remove the false alarms. Choosing helper-free or phase-tolerant decompositions inside the hierarchical pass, using the free pool, would add Qrisp's synthesis gains to QRECLAIM's placement.
3) **Hardware-aware allocation and feed-forward.** Calibrated gate, measurement and reset latencies and coupling-map distance should enter the ERF key. The analyses should also be lifted to control-flow graphs with classically controlled branches, using a lattice join and liveness across branches.

## REFERENCES


[1] B. Bichsel, M. Baader, T. Gehr, and M. Vechev, "Silq: A high-level quantum language with safe uncomputation and intuitive semantics," in *Proc. 41st ACM SIGPLAN Conf. Program. Lang. Design Implement. (PLDI)*, 2020, pp. 286–300.

[2] R. Seidel, N. Tcholtchev, S. Bock, and M. Hauswirth, "Uncomputation in the Qrisp high-level quantum programming framework," in *Proc. Reversible Computation (RC), LNCS vol. 13960*. Springer, 2023, pp. 150–165.

[3] K. Hirata and C. Heunen, "Qurts: Automatic quantum uncomputation by affine types with lifetime," *Proc. ACM Program. Lang.*, vol. 9, no. POPL, pp. 155–182, 2025.

[4] W. K. Wootters and W. H. Zurek, "A single quantum cannot be cloned," *Nature*, vol. 299, no. 5886, pp. 802–803, 1982.

[5] C. H. Bennett, "Logical reversibility of computation," *IBM J. Res. Develop.*, vol. 17, no. 6, pp. 525–532, 1973.

[6] F. Hua, Y. Jin, Y. Chen, S. Vittal, K. Krsulich, L. S. Bishop, J. Lapeyre, A. Javadi-Abhari, and E. Z. Zhang, "CaQR: A compiler-assisted approach for qubit reuse through dynamic circuit," in *Proc. 28th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS), vol. 3*, 2023, pp. 59–71.

[7] M. DeCross, E. Chertkov, M. Kohagen, and M. Foss-Feig, "Qubit-reuse compilation with mid-circuit measurement and reset," *Phys. Rev. X*, vol. 13, no. 4, 2023, art. no. 041057.

[8] A. Paradis, B. Bichsel, S. Steffen, and M. Vechev, "Unqomp: Synthesizing uncomputation in quantum circuits," in *Proc. 42nd ACM SIGPLAN Conf. Program. Lang. Design Implement. (PLDI)*, 2021, pp. 222–236.

[9] A. Paradis, B. Bichsel, and M. Vechev, "Reqomp: Space-constrained uncomputation for quantum circuits," *Quantum*, vol. 8, 2024, art. no. 1258.

[10] Y. Ding, X.-C. Wu, A. Holmes, A. Wiseth, D. Franklin, M. Martonosi, and F. T. Chong, "SQUARE: Strategic quantum ancilla reuse for modular quantum programs via cost-effective uncomputation," in *Proc. ACM/IEEE 47th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2020, pp. 570–583.

[11] H. Jiang, "Qubit recycling revisited," *Proc. ACM Program. Lang.*, vol. 8, no. PLDI, pp. 1264–1287, 2024.

[12] I. Reichental, R. Alon, L. Preminger, M. Vax, and A. Naveh, "Scalable memory recycling for large quantum programs," arXiv:2503.00822, 2025.

[13] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, "Quantum computing with Qiskit," arXiv:2405.08810, 2024.

[14] N. Yu and J. Palsberg, "Quantum abstract interpretation," in *Proc. 42nd ACM SIGPLAN Conf. Program. Lang. Design Implement. (PLDI)*, 2021, pp. 542–558.

[15] M. Poletto and V. Sarkar, "Linear scan register allocation," *ACM Trans. Program. Lang. Syst.*, vol. 21, no. 5, pp. 895–913, 1999.